

CineVerse — Project Vision

1. Overview

CineVerse is a distributed movie ticket booking platform designed to explore and demonstrate real-world backend engineering challenges in highly concurrent and failure-prone environments.

The project focuses on building a system that can maintain correctness and consistency while handling concurrent seat reservations, distributed transactions, asynchronous communication, service failures, and high traffic.

The movie-booking domain is used as the problem space, but the primary objective is to understand and solve distributed systems problems rather than simply building a CRUD-based booking application.

2. Primary Objective

The primary objective of CineVerse is to guarantee that a seat cannot be successfully booked by more than one user, even when thousands of users attempt to reserve the same seat concurrently across multiple application instances.

The system should be able to demonstrate:

```
10,000 concurrent requests
      ↓
Same show
      ↓
Same seat
      ↓
Exactly 1 successful reservation
      ↓
0 double bookings
```

This correctness guarantee must remain valid even when the application is horizontally scaled.

3. Engineering Goals

CineVerse will focus on the following engineering challenges:

3.1 Concurrency and Consistency

Design a seat reservation mechanism that guarantees zero double booking under extreme concurrent access.

Explore and evaluate:

- Optimistic locking
- Pessimistic locking
- Database transactions
- Transaction isolation
- Atomic state transitions
- Redis-based contention reduction

The final implementation should be backed by measurable concurrency tests.

3.2 Distributed Transactions

A movie booking involves multiple independent operations:

```
graph TD; A[Seat Reservation] --> B[Payment]; B --> C[Booking Confirmation]; C --> D[Notification];
```

These operations cannot rely on a single database transaction across services.

CineVerse will implement a Saga-based workflow with compensation to maintain business consistency across distributed services.

The system should correctly handle failures such as:

- Payment failure
 - Payment timeout
 - Booking confirmation failure
 - Refund failure
 - Service crashes
-

3.3 Reliable Event-Driven Communication

CineVerse will use Kafka for asynchronous communication where appropriate.

The system will implement the Transactional Outbox pattern to prevent inconsistencies between database transactions and event publishing.

The system should tolerate temporary Kafka failures without losing business events.

3.4 Idempotency

Distributed systems may deliver requests and events more than once.

CineVerse must ensure that duplicate requests or duplicate events do not create duplicate business effects.

Examples:

Same payment request
↓
One payment

Same Kafka event received multiple times
↓
One business effect

3.5 Resilience

The system should remain reliable when individual components fail.

The project will explore:

- Timeouts
- Retries
- Exponential backoff
- Jitter
- Circuit breakers
- Bulkheads
- Rate limiting

Each resilience mechanism should be introduced to solve a specific failure scenario rather than being added only for technology demonstration.

3.6 High Traffic and Scalability

The system should be able to handle high-concurrency traffic and horizontally scale using Kubernetes.

The project will explore the problem of a sudden traffic spike for a highly popular movie or show.

Possible solutions include:

- Rate limiting
- Controlled admission
- Virtual queues
- Horizontal Pod Autoscaling

The system should be benchmarked under different load patterns.

3.7 Observability

CineVerse should provide enough observability to understand the behaviour of the system in production-like environments.

The platform will include:

- Metrics
- Structured logs
- Distributed traces
- Business-level dashboards
- Infrastructure-level monitoring

An engineer should be able to trace a booking request across multiple services and identify where and why a failure occurred.

3.8 Failure and Chaos Testing

The system will be deliberately tested under failure conditions.

Examples:

```
Redis unavailable
Kafka unavailable
Payment service unavailable
Payment timeout
Booking service crash
Saga orchestrator crash
Kafka consumer crash
Database temporary failure
Network latency
```

The expected recovery behaviour should be documented and verified through automated or repeatable tests.

4. Core System Flow

The primary booking workflow is:

```
User
↓
API Gateway
↓
```

```
Booking Service
  ↓
Inventory Service
  ↓
Seat Hold
  ↓
Payment Service
  ↓
Payment Confirmation
  ↓
Booking Confirmation
  ↓
Kafka
  ↓
Notification Service
```

For failures, the Saga orchestrator coordinates compensating actions.

Example:

```
Payment Successful
  ↓
Booking Confirmation Failed
  ↓
Refund Payment
  ↓
Release Seat
```

5. High-Level Architecture

The initial service boundaries are:

```
API Gateway

Booking Service

Inventory Service

Payment Service

Notification Service

Saga Orchestrator
```

Infrastructure:

```
PostgreSQL
Redis
Kafka
Docker
Kubernetes
Prometheus
Grafana
Loki
OpenTelemetry
Jaeger
k6
```

Technologies may change during development if experiments demonstrate that another approach provides better correctness, performance, reliability, or simplicity.

Technology should never be added only for the purpose of increasing the technology stack.

6. Non-Functional Goals

CineVerse should aim to provide:

Correctness

- Zero double booking
- Valid seat state transitions
- Consistent booking lifecycle
- Idempotent business operations

Reliability

- No loss of committed business events
- Recoverable distributed workflows
- Graceful handling of dependency failures

Scalability

- Horizontal scaling
- Multiple service instances
- Kubernetes-based scaling
- Controlled handling of traffic spikes

Observability

- Metrics
- Logs
- Distributed tracing
- Business and infrastructure dashboards

Performance

Performance will be measured rather than assumed.

The project will record:

- Requests per second
 - P50 latency
 - P95 latency
 - P99 latency
 - Error rate
 - Database load
 - Redis load
 - Kafka consumer lag
-

7. Out of Scope

The following features are intentionally not part of the initial project scope:

- AI-based recommendations
- Movie recommendation engines
- Elasticsearch-based search
- Social features
- Chat functionality
- Complex loyalty programmes
- Real-world payment provider integration
- Production-scale cloud deployment
- Excessive microservice decomposition

The project will prioritise backend engineering depth over feature quantity.

8. Success Criteria

CineVerse will be considered technically successful when the system can demonstrate:

1. 10,000 concurrent requests for the same seat
 - Exactly 1 successful reservation
 - 0 double bookings
2. Multiple application instances
 - Correctness maintained
3. Redis failure
 - System remains correct
4. Kafka failure
 - Business events are not lost

5. Payment failure
 - Saga performs compensation
6. Duplicate events
 - One business effect
7. Duplicate payment requests
 - One payment
8. Service or pod failure
 - System recovers or workflow resumes
9. Kubernetes scaling
 - Correctness maintained
10. Distributed tracing
 - Full booking workflow is traceable
11. Load testing
 - Actual performance characteristics are measured
12. Chaos testing
 - Failure behaviour is understood and documented

9. Engineering Philosophy

CineVerse follows the principle:

Every engineering decision must have a reason.

The project will not be evaluated by the number of frameworks or technologies used.

Instead, it will be evaluated by the ability to answer:

What problem were we solving?

What options did we consider?

Why did we choose this approach?

What are the trade-offs?

What happens when it fails?

How do we know it works?

How did we measure it?

The final project should demonstrate not only the ability to build distributed systems, but also the ability to reason about them.

10. Final Vision

CineVerse is intended to be more than a movie ticket booking application.

It is a practical distributed-systems laboratory where real engineering problems are introduced progressively:

```
graph TD; A[Simple Booking] --> B[Concurrency]; B --> C[Double Booking Problem]; C --> D[Database Consistency]; D --> E[Distributed Booking]; E --> F[Saga]; F --> G[Event Reliability]; G --> H[Outbox]; H --> I[Idempotency]; I --> J[Service Failures]; J --> K[Resilience]; K --> L[High Traffic]; L --> M[Kubernetes Scaling]; M --> N[Observability]; N --> O[Performance Testing]; O --> P[Chaos Testing];
```

Simple Booking
↓
Concurrency
↓
Double Booking Problem
↓
Database Consistency
↓
Distributed Booking
↓
Saga
↓
Event Reliability
↓
Outbox
↓
Idempotency
↓
Service Failures
↓
Resilience
↓
High Traffic
↓
Kubernetes Scaling
↓
Observability
↓
Performance Testing
↓
Chaos Testing

The final goal is to build a system where:

The code demonstrates implementation ability.
The tests demonstrate correctness.
The benchmarks demonstrate scalability.
The failure experiments demonstrate resilience.
The observability demonstrates operational maturity.
The ADRs demonstrate engineering judgement.